



Quantized bounding volume hierarchies for neighbor search in molecular simulations on graphics processing units

Michael P. Howard^{a,*}, Antonia Statt^b, Felix Madutsa^b, Thomas M. Truskett^a,
Athanasios Z. Panagiotopoulos^b

^a McKetta Department of Chemical Engineering, University of Texas at Austin, Austin, TX 78712, United States

^b Department of Chemical and Biological Engineering, Princeton University, Princeton, NJ 08544, United States

ARTICLE INFO

Keywords:

Molecular simulation
Neighbor search
Bounding volume hierarchy
GPU

ABSTRACT

We present an algorithm for neighbor search in molecular simulations on graphics processing units (GPUs) based on bounding volume hierarchies (BVHs). The BVH is compressed into a low-precision, quantized representation to increase the BVH traversal speed compared to a previous implementation. We find that neighbor search using the quantized BVH is roughly two to four times faster than current state-of-the-art methods using uniform grids (cell lists) for a suite of benchmarks for common molecular simulation models. Based on the benchmark results, we recommend using the BVH instead of a single cell list for neighbor list generation in molecular simulations on GPUs.

1. Introduction

Molecular simulation has become a valuable tool to provide insights on microscopic structures and processes, to predict self-assembly and phase behavior, and to rapidly explore parametric design spaces. The wide-spread use of molecular dynamics (MD) simulations [1,2] in computational materials research is due in part to the proliferation of optimized open-source MD software packages [3–6]. The introduction of massively-parallel computing architectures like the graphics processing unit (GPU) significantly expanded the length and time scales accessible in MD simulations [6–10], and most MD packages are now GPU-accelerated to some extent.

Particles in molecular simulations commonly interact through short-ranged, pairwise potentials. The prototypical example is the Lennard-Jones potential,

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad (1)$$

where r is the distance between particles, ϵ is the interaction energy, and σ is the particle diameter. For computational efficiency in MD simulations, Eq. (1) is usually truncated and shifted to zero at a distance r_c that is long enough that the properties of interest are not significantly affected by the neglected attractions. A disordered collection of particles interacting only by Eq. (1) is called a Lennard-Jones fluid.

The calculation of pairwise interactions is usually the most

computationally demanding part of a molecular simulation. The complexity of exhaustively evaluating all pairs from N particles is $O(N^2)$; however, many of these interactions are trivially zero when $r > r_c$. The challenge is then to efficiently determine the subset of particles within a distance r_c of a given particle (Fig. 1a). The pair interactions can be computed directly with these particles, or the subset can be saved to construct a Verlet (neighbor) list [2]; both problems are fundamentally a *neighbor search*.

Neighbor search has historically been performed in molecular simulations using a grid (“cell list”) approach [2]. The grid subdivides space uniformly into cells, and particles are binned into the cells (Fig. 1b). A pairwise distance check between particles is only done for a small number of adjacent cells, reducing the search complexity to $O(Nm)$ with m being the average number of particles in a cell (usually $m \ll N$). This algorithm works well on CPUs [3,11] and has also been successfully adapted to GPUs [8,12,13].

Despite the simplified complexity of the grid search, many distance checks are still wasted by this scheme [11,14]. If the cell size is equal to r_c , the volume of the neighbor-search sphere is only 16% of the explored cell volume in three dimensions. Decreasing the cell size (increasing the number of cells) to reduce this waste results in significant overhead on GPUs [14]. Some alternatives [7,9,10,15] to standard cell lists trade more distance checks for reduced overhead in an attempt to fully leverage the parallelism of the GPU.

In computer gaming and ray tracing, tree structures like bounding

* Corresponding author.

E-mail address: mphoward@utexas.edu (M.P. Howard).

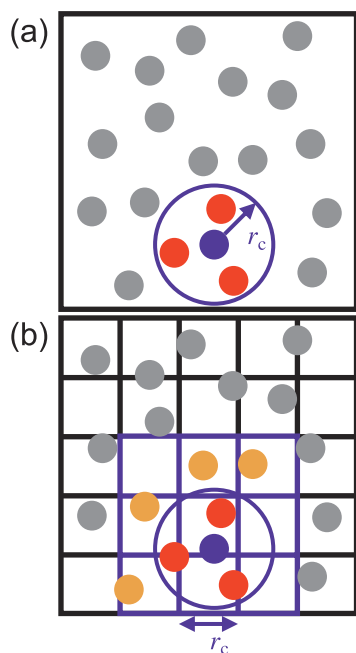


Fig. 1. (a) Illustration of neighbor search. Red particles are neighbors of the blue particle within distance r_c , while the gray particles are not neighbors. (b) Grid algorithm for neighbor search using cell width r_c . The blue particle checks for neighbors from only the blue cells. The red particles are tested and found to be neighbors, the orange particles are tested but are not neighbors, and the gray particles are skipped.

volume hierarchies (BVHs), k -d trees, and octrees are often used instead of a uniform grid to efficiently detect collisions and intersections [16]. These algorithms group particles that are nearby in space into a tree-like hierarchy that can be searched (traversed) by simple intersection tests. Such trees have been successfully employed for neighbor search in molecular simulations on CPUs [17–21].

Recently, some of us proposed using a BVH to perform neighbor search in molecular simulations on GPUs [14]. The BVH significantly outperformed the grid for mixtures having interaction-range disparity, e.g., large colloidal particles dispersed in a solvent, due to its lower traversal overhead for such mixtures. Unsatisfactorily, though, the BVH was often slower than the grid when the size disparity was smaller. The BVH performed worst compared to the grid for a single-component Lennard-Jones fluid; the grid was nearly twice as fast [14]. At the time, we recommended using the BVH for systems having significant interaction range disparity, but the grid approach otherwise.

We have since revisited the BVH neighbor-search algorithm and made substantial improvements to the BVH traversal. The key development is to employ a low-precision, quantized representation that compresses the internal BVH data and reduces the traversal overhead. We performed a comprehensive set of benchmarks for single-component fluids (the previous worst-case scenario) with the new algorithm using recent NVIDIA GPUs. We find that the new BVH algorithm is comparable or superior to the grid method for nearly all benchmark configurations tested. On current GPUs, neighbor search with the BVH is typically two to four times faster than with an equivalent grid algorithm.

The rest of this article is organized as follows. We first provide an overview of the BVH neighbor-search algorithm with a focus on molecular simulations and then describe the improvements we have made to our previous implementation by compressing the BVH (Section 2). We then give details of the algorithm implementation (Section 3) and test its performance for a suite of benchmarks based on representative molecular simulation models for fluids (Section 4). Finally, we summarize our findings and suggest avenues for further investigation

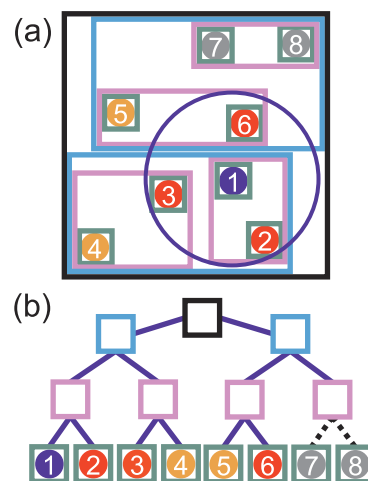


Fig. 2. Schematic bounding volume hierarchy corresponding to configuration of particles inside blue cells of Fig. 1b. (a) Nearby particles (circles) are grouped into successively larger axis-aligned bounding boxes. The leaf nodes are green, the internal nodes are purple and blue, and the root node is black. (b) Hierarchical view of the bounding volumes in (a). Blue solid edges indicate branches that are visited during neighbor search for particle 1 using the shown large circle, while the black dotted edges are skipped. The leaf nodes containing the red particles are tested and found to be neighbors, the leaf nodes containing the orange particles are tested but are not neighbors, and the leaf nodes containing the gray particles are skipped. Note that although the leaf node for particle 1 is trivially visited during traversal, a particle is usually not considered a neighbor of itself.

(Section 5).

2. Algorithm

2.1. Bounding volume hierarchy

A BVH is a tree structure that partitions a system by objects rather than space [16]. A schematic is shown in Fig. 2a for a subset of the particles in Fig. 1. Each object (circles) is placed in a “leaf” node, and leaf nodes are successively merged into internal nodes, ending at the “root” node at the top of the tree. Each node has a bounding volume that encloses all of its descendant objects. In this discussion, we assume that the tree is binary and each internal node has exactly two children, but wider BVHs have also been used for ray-tracing applications [22–24].

Objects within a given volume can be found by traversing the BVH using a binary search [16], illustrated in Fig. 2b. To find neighbors of a given particle, a search volume is tested for overlap with the bounding volume of a node starting from the root, which has the largest bounding volume in the tree and no ancestors. Traversal proceeds to the children of that node if an overlap is detected; otherwise, that branch of the tree can be skipped. Eventually, the traversal reaches a leaf node, where additional (or more expensive) calculations between the search volume and the object in the leaf can be performed if needed.

In order to obtain good traversal performance, objects should be equally balanced between branches of the BVH, and objects that are spatially local should also be grouped nearby in the hierarchy [16]. Typically, there is a tradeoff between the BVH build time and the quality of the BVH for traversal, with the optimal balance depending on the number of traversals performed per build [25]. In our previous work, we constructed a linear bounding volume hierarchy (LBVH) [26] using Karras’s algorithm [27] that builds the BVH from the bottom up to maximize parallelism on the GPU. Higher quality BVHs can be constructed by alternative algorithms [28–30] or the LBVH can be refined with additional processing [25,31], but both carry a significant cost. We have found that the quality of the LBVH is sufficient to obtain

good performance in our benchmarks (Section 4), and so we have not attempted to implement these alternative BVH construction schemes.

We will briefly summarize the algorithm for constructing the LBVBH and traversing it on the GPU for a molecular simulation; additional details and pseudocode are available elsewhere [14,27]. To build the LBVBH, the simulation box is first subdivided into $2^{10} - 1$ bins along each Cartesian axis, with each bin coordinate represented by a 10-bit integer. The particles are assigned into a bin, given a 30-bit Morton code by interleaving the bin coordinates bitwise, and then sorted along a Z-order curve [32]. The Morton codes are subsequently processed to construct a tree hierarchy [27], and bounding volumes are fit for each node by walking up the tree from the leaf nodes. We enclose the nodes in axis-aligned bounding boxes (AABBs) because they are simple to construct, have a small memory footprint, and are easy to test for overlaps. The constructed BVH has $N - 1$ internal nodes for N objects (stored in N leaf nodes), and each internal node has exactly two children [27].

We employ a stackless scheme to traverse the BVH (lines 10–19 of Algorithm 1) [14,28,33,34]. Every particle searches the BVH using a bounding volume centered around the particle; we previously used an AABB with inscribed radius r_c as the search volume [14]. In order to accommodate periodic boundary conditions in molecular simulations, the search volume is translated by appropriate combinations of the lattice vectors, giving 27 total search volumes per particle for a three-dimensional periodic simulation box. When the search volume overlaps an internal node, the traversal descends to the left child of the node. Otherwise, traversal advances along a “rope” to the next node in the tree to test. Traversal terminates when the rope advances past the last node in the tree. We found in preliminary benchmarks that this stackless approach performed favorably compared to using an explicitly-managed stack [35,36].

2.2. Compressed BVH

The total neighbor search time in our previous implementation was dominated by the BVH traversal. Moreover, a significant fraction of the BVH build time was simply due to sorting the Morton codes. We accordingly focused our efforts on improving the traversal. In particular, inspired by developments in ray tracing [24,37–41], we considered ways to compress the BVH to reduce the data loaded per node, which is often a bottleneck for GPUs.

The original memory layout of our BVH nodes [14] is shown in Fig. 3a with slight modification. An AABB is defined by a lower and upper bound in three dimensions, represented as floating-point values. Each node additionally stores the integer index of its left child in the

tree and its skip rope for traversal. When the node is a leaf, it has no left child, and the node instead stores the index of the object in the leaf. In practice, this distinction is made by representing the indexes as 32-bit signed integers, with nodes being positive and objects being negative.

Many MD simulations are conducted using 64-bit (“double”) floating-point precision for the particle data and in key steps to ensure faithful integration of the equations of motion [8]. However, such high precision would be wasteful for the AABB since only simple overlap tests are needed. For smaller memory demands and better performance, the AABB bounds can instead be stored in lower precision, e.g., 32-bit (“single”) floating-point precision. A node with a single-precision AABB is then only 32 bytes (Fig. 3a) instead of the minimum of 56 bytes that would be needed for a node with a double-precision AABB.

For correctness, the lower-precision AABB bounds must fully enclose the volume of the higher-precision AABB or object [41]. For example, to fit a single-precision AABB to a point particle represented in double precision, the lower bound of the AABB is computed by rounding down the particle coordinates to the nearest representable single-precision value, while the upper bound is computed by rounding up. The AABB then represents the particle position with uncertainty due to the loss of precision. Any search volume overlapping the point particle must also overlap the AABB, and so correctness of the search is ensured (no false negatives). However, there may be some overlaps with the AABB that do not correspond to actual overlaps with the particle (false positives).

We propose a scheme to further compress the node memory beyond single-precision AABB bounds through appropriate discretization of space into bins. The AABB bounds can then be stored in a compact, quantized integer representation using the bin coordinates (Fig. 3c). This strategy is similar to previous ones [24,37,40,41] but uses a global coordinate system based on the root-node AABB that is better suited to binary trees and stackless traversal. We subdivide the root-node AABB into $2^{10} - 1$ bins along each dimension. The lower and upper bounds of the AABBs in the BVH are snapped onto this grid, ensuring that lower bounds are rounded down and upper bounds are rounded up for correctness [24,41]. These 10-bit integer representations can then be concatenated into a 32-bit integer representing each bound (2 bits are unused). The entire node needs only 16 bytes (Fig. 3b), half the size of the node with a single-precision AABB.

More false-positive overlaps are expected for the quantized AABB than for the single-precision or double-precision AABB, particularly when the discretization becomes large relative to r_c . The number of false overlaps affects traversal performance, and so there is a tradeoff between reducing the size of the node in this way and the BVH quality for traversal. For typical simulation boxes of size 50σ , the global discretization is roughly 0.05σ , leading to loss of precision in the first decimal place. We will characterize the effectiveness of this discretization later in our benchmarks.

2.3. Additional improvements

In order to fully capitalize on the compressed BVH representation, we have further modified our original BVH traversal algorithm [14], presented in its new form in Algorithm 1, so that no additional distance checks are performed between particles once a leaf node is reached. Instead, we simply accept particles as neighbors if the search volume overlaps the leaf in the traversal (line 14). With this approach, the particle coordinates, which are typically stored in higher precision than the AABB, no longer need to be loaded. However, a small number of false-positive neighbors will be identified, which can be filtered or rejected in a later stage if desired.

Algorithm 1 BVH neighbor search in a molecular simulation.

1: $\{v_j\} \leftarrow$ periodic image translation vectors

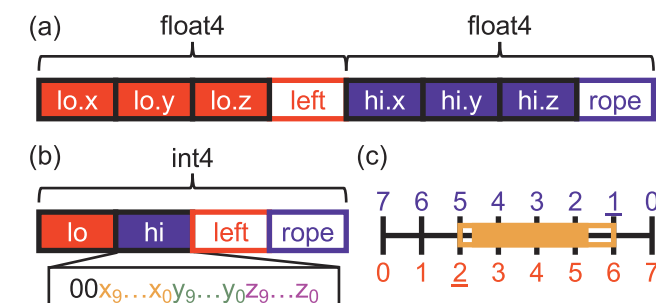


Fig. 3. (a) BVH node layout similar to Ref. [14]. lo and hi are lower and upper bounds of the AABB, while left and rope are indexes of nodes for traversal. (b) Compressed node layout for quantized bounds that requires only half the memory of the original layout (a). The box shows how the quantized bounds are concatenated into one 32-bit integer. (c) Schematic illustration of quantized lower (red) and upper (blue) bounds in x for an AABB (filled orange). The underlined indexes give the integer bounds of the quantized AABB (open orange) on the discretized grid (black). float4 and int4 are CUDA data types that can be used to store the data, with each being 16 bytes [42].

Algorithm 1 BVH neighbor search in a molecular simulation.

```

2: for each particle  $i$  in parallel
3:    $\mathbf{x}_i \leftarrow$  position of particle  $i$ 
4:    $b \leftarrow$  integer with bit  $j$  set to 1 if  $\mathbf{x}_i + \mathbf{v}_j$  overlaps the root node
5:   while  $b \neq 0$  do
6:      $j \leftarrow$  index of next set bit in  $b$ 
7:     Set bit  $j$  in  $b$  to 0.
8:      $S \leftarrow \text{SPHERE}(\mathbf{x}_i + \mathbf{v}_j, r_c)$ 
9:      $n \leftarrow$  root node
10:    while untested node  $n$  remains do
11:       $A \leftarrow \text{AABB}(n)$ 
12:      if  $S$  overlaps  $A$  then
13:        if  $n$  is a leaf then
14:          Process object in  $n$  as a neighbor.
15:         $n \leftarrow$  next node to test from rope
16:      else
17:         $n \leftarrow$  left child of  $n$ 
18:      else
19:         $n \leftarrow$  next node to test from rope

```

In order to limit the number of false-positive overlaps, we search the BVH using a spherical volume with radius r_c centered around each particle rather than the AABB with inscribed radius r_c from our previous work [14] (line 8). Using a sphere instead of an AABB decreases the search volume by roughly 50%, which reduces the number of false positives from, e.g., overlaps between corners of AABBs. To perform the sphere–AABB overlap tests with internal and leaf nodes, the quantized AABB is first decompressed into a single-precision floating-point representation (line 11). Rounding ensures that the node bounds are correctly reconstructed. For best arithmetic instruction performance, the sphere is also represented in single precision regardless of the precision of the particle data; the sphere radius is padded to account for uncertainty from loss of precision when the particle data is double precision.

Overlap is then tested by finding the point $\mathbf{y}$ in the AABB nearest to the center of the sphere $\mathbf{x}$ (line 12). If $\mathbf{a}$ and $\mathbf{b}$ are the lower and upper bounds of the AABB, then the nearest point to $\mathbf{x}$ in the AABB is [16]

$$\mathbf{y} = \min(\max(\mathbf{x}, \mathbf{a}), \mathbf{b}), \quad (2)$$

where the minimum and maximum operations are applied component-wise. The sphere and the AABB overlap if the distance from the center of the sphere to this point is less than the sphere radius, i.e., $|\mathbf{y} - \mathbf{x}| \leq r_c$.

The search sphere is translated by appropriate combinations of the simulation box lattice vectors to account for periodic boundary conditions (line 8). We previously implemented this as a loop over images [14], but this scheme may lead to execution divergence on the GPU when some images do not overlap the BVH. As a minor optimization, we now test all images for overlap with the root node and construct a bitset indicating which images overlap before beginning traversal (line 4). The next image traversed for a particle can be selected from this bitset (line 6), skipping over nonoverlapping images, without wasting iterations of the loop.

Finally, our previous implementation of the BVH algorithm lumped multiple particles into leaf nodes using the Z-order curve before the BVH was constructed [14]. This procedure reduced the depth of the BVH for traversal, and we found that grouping 4 consecutive particles from the Z-order curve per leaf was optimal in that implementation. However, subsequent analysis of the BVH structure revealed that this procedure produced rare large AABBs when a grouping of 4 spanned a jump in the Z-order curve. The AABBs of internal nodes enclosing such leaf nodes were correspondingly large, slowing traversal. A better approach would first construct the tree with one particle per leaf and then collapse subtrees according to an appropriate heuristic [25]. In practice, we have found in our benchmarks that the BVH performance is already good without any subtree collapse and leave this as a possible future optimization.

3. Implementation

We implemented the quantized BVH algorithm in a neighbor-search library [43] that uses HOOMD-blue (version 2.4.1) [6,12] as a dependency. HOOMD-blue is an open-source molecular dynamics package optimized for NVIDIA GPUs. The remainder of our discussion will accordingly use terminology from the CUDA programming model [42]. HOOMD-blue can be configured to use either single- or double-precision floating-point values to represent particles, and our implementation supports both data types.

We constructed an LVBH using Karras’s algorithm [27] with the CUB library [44] for sorting. Neighbor search was performed using Algorithm 1 with the particles in Z-order so that threads within the same CUDA warp traversed similar parts of the tree [14]. Arithmetic operations for compressing and decompressing the AABBs and detecting overlaps were performed using IEEE 754-2008 round-down and round-up modes [45] through the appropriate CUDA intrinsics [42].

We compared the neighbor-search performance of the BVH to an equivalent uniform grid algorithm. To construct the grid, particles were binned into cells of size r_c [14], and the particles in each cell were identified by sorting [8]. This approach differs from the current implementation in HOOMD-blue [12], where particles are inserted into cells using limited atomic operations. The sorting approach produces a more memory-compact list of particles in cells than the atomic operations, and the particle list is additionally deterministic if the sorting algorithm is stable. Both are desirable for molecular simulations and are also properties shared by the LVBH algorithm. We found in selected benchmarks that the sorting approach with compact memory was roughly two times slower than HOOMD-blue’s aggressively optimized atomic approach; this appears to partially be an inherent cost of making an equivalent grid algorithm that is both deterministic and minimally memory consuming like the LVBH.

Grid traversal [12,14,46] was implemented using n threads per search volume, where n is a power of 2 smaller than the CUDA warp size. The n threads cooperatively process particles from adjacent cells with a stencil. They communicate using CUDA shuffle instructions [42] that allow threads within a warp to efficiently read each others’ registers. Particle data was sorted into the same order as the grid to improve traversal speeds, and neighbor search was performed in this sorted order. All distance evaluations were performed using the precision of the particle data (single or double precision).

4. Performance

We tested the performance of both algorithms using disordered (fluid) configurations generated from MD simulations of commonly used models (see below). We emphasize that these benchmarks gauge the performance of the neighbor search algorithm itself, which is part of the total molecular simulation time along with, e.g., pair force evaluation and integration of the equations of motion. We collected 10 independent configurations for each benchmark from constant-temperature, constant-volume MD simulations in HOOMD-blue. The simulation time step was 0.005τ , where $\tau = \sqrt{m\sigma^2/\epsilon}$ is the unit of time for particles of mass m . The temperature T was controlled using a Langevin thermostat with friction coefficient $0.1m/\tau$ [47]. The simulation box was cubic and periodic in all three dimensions, and its size L was set to obtain the desired density $\rho = N/L^3$ for N particles.

We subsequently determined the average time to build and traverse a BVH or grid using these configurations. The traversal counted the number of particles within a distance r_c of each of the N particles in the configuration, subject to the periodic boundary conditions. We first ran 200 builds and traversals to allow runtime autotuners [12] to determine the optimal CUDA kernel launch parameters. We then disabled the autotuners and measured the average build and traversal times over 500 iterations. We repeated this measurement 5 times, determined the

median value, and averaged it for all configurations. We ran all benchmarks using both single- and double-precision particle data representations.

The benchmarks were performed on recent NVIDIA Tesla GPUs commonly found in supercomputing centers: K80, P100 for PCIe (16 GB), and V100 for PCIe (32 GB). We additionally tested performance on GeForce GTX 1080, which is designed for computer gaming but is also often used for computations. Each GPU architecture has unique features that are beyond the scope of this article to describe [42]. Benchmark programs for K80, P100, and V100 were compiled using CUDA 9.0 and GCC 6.4.0, while the GTX 1080 benchmark used CUDA 8.0 and GCC 4.8.4. The benchmarks were run on a single GPU driven by a single process; the algorithms presented here can be straightforwardly generalized to execute on multiple GPUs through a domain decomposition strategy [12].

4.1. Lennard-Jones fluid

We first tested the performance of the algorithms for the Lennard-Jones fluid, which presented the most significant challenges for our original BVH neighbor-search implementation [14]. We ran benchmarks with $r_c = 3.0\sigma$ at both low ($\rho = 0.2/\sigma^3$) and high ($\rho = 0.8/\sigma^3$) densities to mimic a range of typical simulation conditions. (The triple-point density of the Lennard-Jones fluid is roughly $0.86/\sigma^3$ [48].) Configurations with $N = 128,000$ particles were sampled every $10^4\tau$ at temperature $T = 1.5\epsilon/k_B$ (above the critical temperature [49,50]), where k_B is Boltzmann's constant. This number of particles was sufficient to fully saturate the GPU with work in our simulations and benchmarks.

When the particle data was represented in double-precision, the BVH neighbor search was consistently faster than the grid for all GPUs tested (Fig. 4). The time required to build the LBVBH was nearly twice the time to construct the grid, which can be attributed to the additional overhead of building the tree hierarchy and fitting the bounding volumes after sorting the particles. However, the traversal of the LBVBH was much faster than for the grid, resulting in a significant net speedup

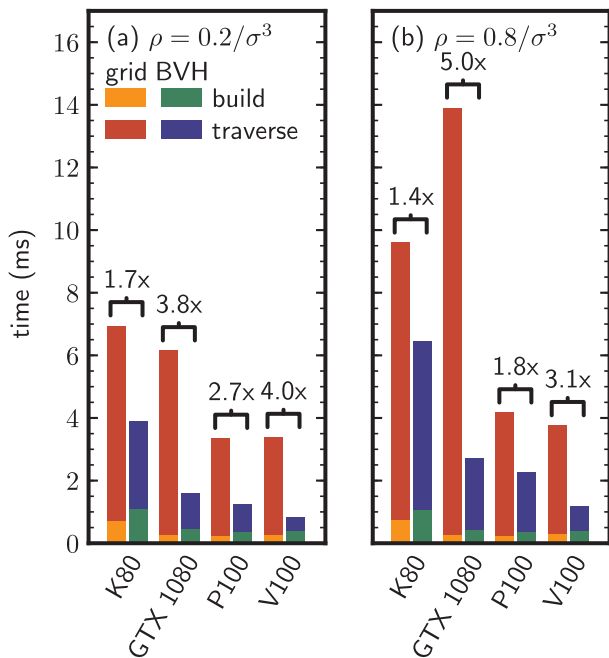


Fig. 4. Lennard-Jones fluid neighbor-search benchmark for $N = 128,000$ particles represented in double precision at (a) $\rho = 0.2/\sigma^3$ and (b) $\rho = 0.8/\sigma^3$ with $T = 1.5\epsilon/k_B$. The traversal counted the number of neighbors within a cutoff $r_c = 3.0\sigma$. Speedups are the ratios of the total times rounded down to the nearest tenth.

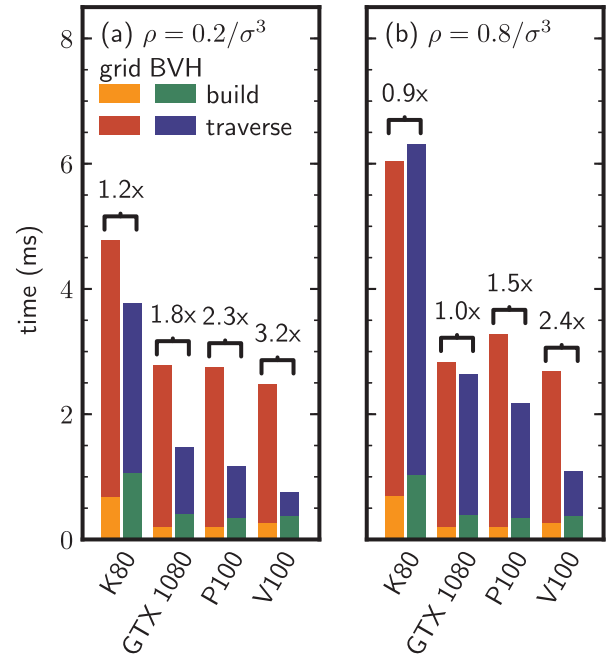


Fig. 5. Lennard-Jones fluid neighbor-search benchmark for $N = 128,000$ particles represented in single precision at (a) $\rho = 0.2/\sigma^3$ and (b) $\rho = 0.8/\sigma^3$ with $T = 1.5\epsilon/k_B$. The traversal counted the number of neighbors within a cutoff $r_c = 3.0\sigma$. Speedups are the ratios of the total times rounded down to the nearest tenth.

of the neighbor search. The speedup on the Tesla GPUs was larger at the lower density, for which the overhead of accessing a cell from the grid is higher (cell occupancy is lower). However, the largest speedup was obtained for the GTX 1080 at $\rho = 0.8/\sigma^3$ due to the low double-precision arithmetic instruction throughput of that GPU [42], which limited the grid algorithm performance. In such cases, using the quantized BVHs with single-precision arithmetic is highly advantageous.

We repeated the same benchmark using single-precision particle data. We expected the grid performance to be more sensitive to the precision of the particle data than the BVH because the grid does not use a low-precision internal representation. Consistent with this expectation, the BVH neighbor-search performance was again comparable or superior to the grid (Fig. 5), but the speedups were generally smaller than for the double-precision data (Fig. 4). This is likely due to two reasons: (1) the single-precision arithmetic throughput of the GPU is higher than the double-precision throughput, resulting in faster grid traversal, and (2) the quantized BVH node is no longer smaller than the single-precision particle data; both are 16 bytes in our implementation. The first effect is most pronounced for the GTX 1080 when comparing Figs. 4 and 5, while the second affects all GPUs tested. However, we emphasize that on recent GPUs (P100, V100), the BVH neighbor search is still faster than the grid even for single-precision particle data.

The previous benchmarks were run with N sufficiently large that the GPU was fully saturated with work. In smaller simulations or to obtain efficient strong scaling in multi-GPU simulations [12], it is important to have good performance even when the number of particles becomes small. For the grid, this is aided by assigning n CUDA threads to cooperatively process the neighbors of each particle [12]. The optimal value of n depends on N , ρ , and the GPU, and can be determined at run time. On the other hand, the BVH traversal (Algorithm 1) uses only one thread per particle regardless of N and ρ , which could fail to saturate the GPU with work at small N .

We generated additional configurations for the Lennard-Jones fluid at $\rho = 0.6/\sigma^3$ with N ranging from 1000 up to 128,000. We determined the optimal n for the grid traversal and report only the fastest total time. Fig. 6 shows the results on the Tesla V100 with double-precision

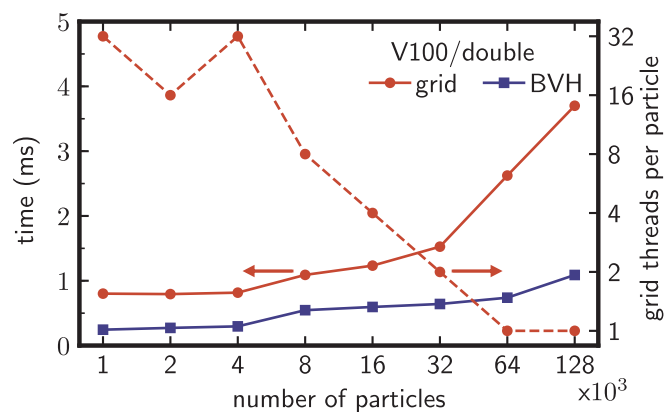


Fig. 6. Lennard-Jones fluid benchmark on Tesla V100 for PCIe (32 GB) for N particles represented in double precision at density $\rho = 0.6/\sigma^3$ and $T = 1.5\epsilon/k_B$. The traversal counted the number of neighbors within a cutoff $r_c = 3.0\sigma$. For the grid, different numbers of threads n per particle were tested; the fastest time is reported on the left axis with the corresponding number of threads on the right axis.

particle data. (Qualitatively similar results were obtained for most other benchmark configurations.) As expected, the optimal n for the grid increased as N decreased. Nonetheless, the BVH consistently outperformed the grid for all N , despite the effectively wider parallelism of the grid traversal. This indicates that the BVH algorithm is well-suited even for multi-GPU simulations where strong scaling efficiency is needed.

The quantized BVHs provide superior performance to the grid, especially with double-precision particle data, due to their decreased memory traffic and the higher throughput of single-precision arithmetic instructions. However, compressing the node using integers reduces the tightness of the AABB bounds and may introduce additional “false-positive” neighbors. Although this does not affect the correctness of the neighbor search (no true neighbors are missed), it may increase the cost of other steps in a molecular simulation, e.g., force calculation from a neighbor list, and it is desirable to keep the number of false-positive neighbors small.

We counted the average number of neighbors per particle found with the quantized BVH, and compared it to the “true” number that lie within r_c as determined by the grid. We found for the Lennard-Jones fluid that on average the quantized BVH identified 1.5 false-positive neighbors per particle at $\rho = 0.2/\sigma^3$ and 3.8 false-positive neighbors per particle at $\rho = 0.8/\sigma^3$, corresponding to a roughly 5% increase in the total number of neighbors. This small number of false positives could be removed by additional processing, or can simply be accepted provided that subsequent calculations using the neighbors are not too computationally intensive.

Overall, the quantized BVH gave a speedup of $1.7\times$ to $3.1\times$ compared to the previous BVH algorithm [14] and $1.1\times$ to $2.3\times$ compared to HOOMD-blue’s atomic-operation grid algorithm on Tesla V100. (Although the latter speedup is smaller than in Figs. 4 and 5, the quantized BVH has the additional benefits of consuming less memory and producing a deterministic neighbor list.) To contextualize these values, we profiled the contribution of the neighbor search to the total simulation time using HOOMD-blue’s current algorithms with an optimized neighbor-list buffer width. The neighbor search contribution ranged from as low as 20% for the grid at $\rho = 0.2/\sigma^3$ to nearly 50% for the BVH at $\rho = 0.8/\sigma^3$, meaning that the quantized BVH could speedup the total simulation by as much as $1.5\times$ on Tesla V100. In practice, though, the actual speedup may be smaller due to additional overhead in the simulations and package-specific implementation details, which are not present in our benchmarks.

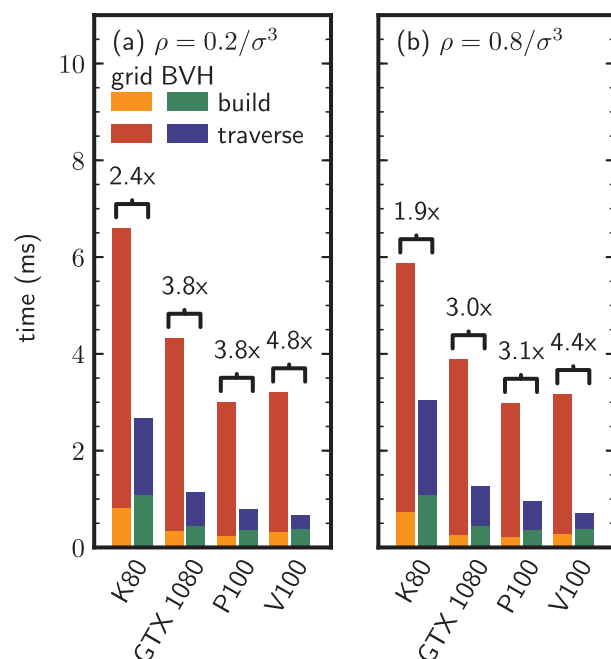


Fig. 7. WCA fluid neighbor-search benchmark for $N = 128,000$ particles represented in double precision at (a) $\rho = 0.2/\sigma^3$ and (b) $\rho = 0.8/\sigma^3$ with $T = 1.5\epsilon/k_B$. The traversal counted the number of neighbors within a cutoff $r_c = 2^{1/6}\sigma$. Speedups are the ratios of the total times rounded down to the nearest tenth.

4.2. Weeks–Chandler–Andersen fluid

Although the Lennard-Jones fluid is a standard simulation benchmark, many simulation models have shorter interaction ranges than $r_c = 3.0\sigma$. A common example is the Weeks–Chandler–Andersen (WCA) fluid [51], which is a model for nearly hard spheres obtained by truncating and shifting Eq. 1 to zero at its minimum ($r_c = 2^{1/6}\sigma$). Such short cutoffs are often used in fluids with very short-ranged or purely repulsive interactions, but may prove challenging for the grid neighbor search for two reasons. First, if the grid cell size is equal to r_c , the cell occupancy becomes small as r_c decreases, and the overhead of accessing a cell increases. Second, the number of cells in the grid may grow significantly as r_c decreases, limiting the volume of the simulation box that can be searched. The BVH suffers from neither of these issues because it partitions the system based on objects rather than space.

We generated configurations of the WCA fluid with the same N , T , and ρ as for the Lennard-Jones fluid. Fig. 7 shows the neighbor search times for the double-precision particle data. The BVH outperformed the grid by a larger factor than for the Lennard-Jones fluid (Fig. 4), consistent with our expectations that the grid should be less favorable with a shorter cutoff radius. However, the speedup on GTX 1080 at the higher density (Fig. 7b) was smaller than for the Lennard-Jones fluid (Fig. 4b) due to the lower cell occupancy, which resulted in fewer direct distance evaluations per particle for the grid. Similar trends were obtained for the WCA fluid as for the Lennard-Jones fluid using single-precision particle data instead of double-precision data, and so these results are omitted here for brevity.

4.3. Spinodal decomposition

We finally tested for effects of inhomogeneity often encountered in self-assembly or phase coexistence simulations, where it is common to have regions of higher or lower density within one simulation box. The BVH should adapt well to such density variations because it partitions by objects rather than space, and so is expected to again perform favorably compared to the grid. To generate configurations with density

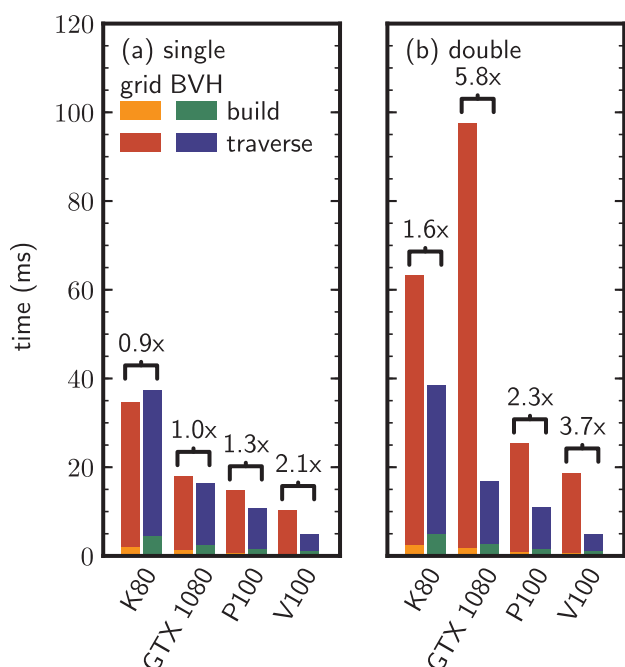


Fig. 8. Spinodal decomposition benchmark for $N = 10^6$ particles represented in (a) single and (b) double precision. The fluid initially had $\rho = 0.2/\sigma^3$ and $T = 1.5\epsilon/k_B$. The temperature was quenched to $T = 0.8\epsilon/k_B$, below the critical point of the Lennard-Jones fluid [49,50], and the fluid decomposed into liquid and vapor regions. The traversal counted the number of neighbors within a cutoff $r_c = 3.0\sigma$. Speedups are the ratios of the total times rounded down to the nearest tenth.

variations, we quenched the Lennard-Jones fluid into the spinodal region of its phase diagram, where it spontaneously decomposes into regions of low density (vapor) and high density (liquid). We first equilibrated a supercritical fluid of $N = 10^6$ particles at $\rho = 0.2/\sigma^3$ and $T = 1.5\epsilon/k_B$. We then reduced the temperature below the critical point [49,50] to $T = 0.8\epsilon/k_B$ and sampled configurations every $10^3\tau$ as the fluid underwent spinodal decomposition. In addition to testing the impact of density variations, this benchmark also assesses BVH performance for large N .

Fig. 8 reports the neighbor search times for both single-precision and double-precision particle data. We again find that the BVH consistently outperformed the grid for nearly all GPUs and benchmark configurations. The largest speedups were obtained for double-precision particle data (Fig. 8b), where the quantized BVHs are most advantageous due to the smaller node size compared to the particle data and the higher single-precision arithmetic instruction throughput. However, good speedups were also obtained for the single-precision particle data, again rendering the BVH the best choice for neighbor search in such systems.

5. Conclusions

We developed an improved implementation of our previously proposed algorithm [14] for performing neighbor searches with BVHs in molecular simulations using GPUs. A low-precision, quantized representation of the BVH nodes significantly increased the BVH traversal speed. We showed with a suite of benchmarks that the improved BVH neighbor search outperforms an equivalent grid (“cell list”) search on multiple generations of NVIDIA GPUs, and is roughly two to four times faster on current GPUs. Given these benchmark results for single-component fluids and our previous finding that BVHs outperform a single cell list for size-asymmetric pair interactions in mixtures [14], we recommend using the BVH instead of a single cell list to generate neighbor lists in molecular simulations on GPUs.

Looking forward, the performance of the BVH neighbor search may be further improved by advances in software and hardware. Given that the LBVH build time is still only a small fraction of the total neighbor-search time, higher quality BVHs may improve the neighbor search performance. However, the longer times needed to build such BVHs must be offset by significantly faster traversal. The strategies of Ref. [15] may be combined with the BVH as a form of subtree collapse to reduce the number of traversals needed and to maximize parallelism. A recently introduced NVIDIA GPU has specialized hardware for ray tracing, with dedicated units for BVH traversal of rays against triangles in leafs [52]. It may be possible to capitalize on this hardware to perform neighbor searches in molecular simulations, possibly utilizing the free parts of the GPU to overlap other calculations, if it can be programmed for more general purposes.

Data availability

The raw and processed data required to reproduce these findings is available from the corresponding author (M.P.H.) upon request. The implementations of the algorithms benchmarked in this work are available as open-source software [43].

CRediT authorship contribution statement

Michael P. Howard: Conceptualization, Methodology, Software, Investigation, Visualization, Writing - original draft. **Antonia Statt:** Methodology, Investigation, Validation, Writing - review & editing. **Felix Madutsa:** Conceptualization, Investigation, Writing - review & editing. **Thomas M. Truskett:** Supervision, Funding acquisition, Writing - review & editing. **Athanasios Z. Panagiotopoulos:** Conceptualization, Supervision, Funding acquisition, Writing - review & editing.

Acknowledgments

We gratefully acknowledge the NVIDIA Corporation for providing access to the PSG Cluster to perform the benchmarks on Tesla K80, P100, and V100. M.P.H. and T.M.T. acknowledge support from the Welch Foundation (Grant No. F-1696). Financial support for this work (A.S., A.Z.P.) was partially provided by the Princeton Center for Complex Materials, a U.S. National Science Foundation Materials Research Science and Engineering Center (Award No. DMR-1420541).

References

- [1] M.P. Allen, D.J. Tildesley, *Computer Simulation of Liquids*, Oxford University Press, New York, 1991.
- [2] D. Frenkel, B. Smit, *Understanding Molecular Simulation*, second ed., Academic Press, San Diego, 2002.
- [3] S. Plimpton, Fast Parallel Algorithms for Short-Range Molecular Dynamics, *J. Comput. Phys.* 117 (1) (1995) 1–19, <https://doi.org/10.1006/jcph.1995.1039>.
- [4] J.C. Phillips, R. Braun, W. Wang, J. Gumbart, E. Tajkhorshid, E. Villa, C. Chipot, R.D. Skeel, L. Kalé, K. Schulten, Scalable Molecular Dynamics with NAMD, *J. Comput. Chem.* 26 (16) (2005) 1781–1802, <https://doi.org/10.1002/jcc.20289>.
- [5] B. Hess, C. Kutzner, D. van der Spoel, E. Lindahl, GROMACS 4: Algorithms for Highly Efficient, Load-Balanced, and Scalable Molecular Simulation, *J. Chem. Theory Comput.* 4 (3) (2008) 435–447, <https://doi.org/10.1021/ct700301q>.
- [6] J.A. Anderson, C.D. Lorenz, A. Travesset, General purpose molecular dynamics simulations fully implemented on graphics processing units, *J. Comput. Phys.* 227 (10) (2008) 5342–5359, <https://doi.org/10.1016/j.jcp.2008.01.047>.
- [7] M.S. Friedrichs, P. Eastman, V. Vaidyanathan, M. Houston, S. LeGrand, A.L. Beberg, D.L. Ensign, C.M. Bruns, V.S. Pande, Accelerating Molecular Dynamic Simulation on Graphics Processing Units, *J. Comput. Chem.* 30 (6) (2009) 864–872, <https://doi.org/10.1002/jcc.21209>.
- [8] P.H. Colberg, F. Höfling, Highly accelerated simulations of glassy dynamics using GPUs: Caveats on limited floating-point precision, *Comput. Phys. Commun.* 182 (5) (2011) 1120–1129, <https://doi.org/10.1016/j.cpc.2011.01.009>.
- [9] A.W. Götz, M.J. Williamson, D. Xu, D. Poole, S. Le Grand, R.C. Walker, Routine Microsecond Molecular Dynamics Simulations with AMBER on GPUs. 1. Generalized Born, *J. Chem. Theory Comput.* 8 (5) (2012) 1542–1555, <https://doi.org/10.1021/ct200909j>.
- [10] M.J. Abraham, T. Murtola, R. Schulz, S. Páll, J.C. Smith, B. Hess, E. Lindahl,

- GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, *Software X* 1–2 (2015) 19–25, <https://doi.org/10.1016/j.softx.2015.06.001>.
- [11] P.J. in't Veld, S.J. Plimpton, G.S. Grest, Accurate and efficient methods for modeling colloidal mixtures in an explicit solvent using molecular dynamics, *Comput. Phys. Commun.* 179 (5) (2008) 320–329, <https://doi.org/10.1016/j.cpc.2008.03.005>.
 - [12] J. Glaser, T.D. Nguyen, J.A. Anderson, P. Lui, F. Spiga, J.A. Millan, D.C. Morse, S.C. Glotzer, Strong scaling of general-purpose molecular dynamics simulations on GPUs, *Comput. Phys. Commun.* 192 (2015) 97–107, <https://doi.org/10.1016/j.cpc.2015.02.028>.
 - [13] K. Rushaidat, L. Schwiebert, B. Jackman, J. Mick, J. Potoff, Evaluation of Hybrid Parallel Cell List Algorithms For Monte Carlo Simulation, 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems, 2015, pp. 1859–1864, <https://doi.org/10.1109/HPCC-CSS-ICESS.2015.260>.
 - [14] M.P. Howard, J.A. Anderson, A. Nikoubashman, S.C. Glotzer, A.Z. Panagiotopoulos, Efficient neighbor list calculation for molecular simulation of colloidal systems using graphics processing units, *Comput. Phys. Commun.* 203 (2016) 45–52, <https://doi.org/10.1016/j.cpc.2016.02.003>.
 - [15] S. Páll, B. Hess, A flexible algorithm for calculating pair interactions on SIMD architectures, *Comput. Phys. Commun.* 184 (12) (2013) 2641–2650, <https://doi.org/10.1016/j.cpc.2013.06.003>.
 - [16] C. Ericson, *Real-Time Collision Detection*, Elsevier Science, New York, 2004.
 - [17] S. Grudin, S. Redon, Practical Modeling of Molecular Systems with Symmetries, *J. Comput. Chem.* 31 (9) (2010) 1799–1814, <https://doi.org/10.1002/jcc.21434>.
 - [18] S. Artemova, S. Grudin, S. Redon, A Comparison of Neighbor Search Algorithms for Large Rigid Molecules, *J. Comput. Chem.* 32 (13) (2011) 2865–2877, <https://doi.org/10.1002/jcc.21868>.
 - [19] J.A. Anderson, M.E. Irrgang, S.C. Glotzer, Scalable Metropolis Monte Carlo for simulation of hard shapes, *Comput. Phys. Commun.* 204 (2016) 21–30, <https://doi.org/10.1016/j.cpc.2016.02.024>.
 - [20] M.M.C. Tortora, J.P.K. Doye, Hierarchical bounding structures for efficient virial computations: Towards a realistic molecular description of cholesterics, *J. Chem. Phys.* 147 (22) (2017) 224504, <https://doi.org/10.1063/1.5002666>.
 - [21] Q.P. Chen, B. Xue, J.I. Siepmann, Using the k-d Tree Data Structure to Accelerate Monte Carlo Simulations, *J. Chem. Theory Comput.* 13 (4) (2017) 1556–1565, <https://doi.org/10.1021/acs.jctc.6b01222>.
 - [22] H. Dammert, J. Hanika, A. Keller, Shallow Bounding Volume Hierarchies for Fast SIMD Ray Tracing of Incoherent Rays, *Comput. Graphics Forum* 28 (4) (2008) 1225–1233, <https://doi.org/10.1111/j.1467-8659.2008.01261.x>.
 - [23] M. Ernst, G. Greiner, Multi bounding volume hierarchies, 2008 IEEE Symposium on Interactive Ray Tracing, 2008, pp. 35–40, <https://doi.org/10.1109/RT.2008.4634618>.
 - [24] H. Ylité, T. Karras, S. Laine, Efficient Incoherent Ray Traversal on GPUs Through Compressed Wide BVHs, *Proceedings of High Performance Graphics (HPG '17)*, 2017, p. 4, <https://doi.org/10.1145/3105762.3105773>.
 - [25] T. Karras, T. Aila, Fast Parallel Construction of High-Quality Bounding Volume Hierarchies, *Proceedings of the 5th High-Performance Graphics Conference (HPG '13)*, 2013, pp. 89–99, <https://doi.org/10.1145/2492045.2492055>.
 - [26] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, D. Manocha, Fast BVH Construction on GPUs, *Comput. Graphics Forum* 28 (2) (2009) 375–384, <https://doi.org/10.1111/j.1467-8659.2009.01377.x>.
 - [27] T. Karras, Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees, *Proceedings of the Fourth ACM SIGGRAPH/ Eurographics Conference on High-Performance Graphics (EGGH-HPG'12)*, 2012, pp. 33–37.
 - [28] J.D. MacDonald, K.S. Booth, Heuristics for ray tracing using space subdivision, *Visual Comput.* 6 (3) (1990) 153–166, <https://doi.org/10.1007/BF01911006>.
 - [29] M. Stich, H. Friedrich, A. Dietrich, Spatial Splits in Bounding Volume Hierarchies, *Proceedings of the Conference on High Performance Graphics 2009 (HPG '09)*, 2009, pp. 7–13, <https://doi.org/10.1145/1572769.1572771>.
 - [30] K. Garanzha, J. Pantaleoni, D. McAllister, Simpler and Faster HLBVH with Work Queues, *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics (HPG '11)*, 2011, pp. 59–64, <https://doi.org/10.1145/2018323.2018333>.
 - [31] L.R. Domingues, H. Pedrini, Bounding Volume Hierarchy Optimization Through Agglomerative Treelet Restructuring, *Proceedings of the 7th Conference on High-Performance Graphics (HPG '15)*, 2015, pp. 13–20, <https://doi.org/10.1145/2790060.2790065>.
 - [32] G.M. Morton, A Computer Oriented Geodetic Data Base; and A New Technique in File Sequencing, IBM Corporation, 1966 Tech. Rep.
 - [33] B. Smits, Efficiency Issues for Ray Tracing, *J. Graph. Tools* 3 (2) (1998) 1–14, <https://doi.org/10.1080/10867651.1998.10487488>.
 - [34] R. Torres, P.J. Martín, A. Gavilanes, Ray Casting using a Roped BVH with CUDA, *Proceedings of the 25th Spring Conference on Computer Graphics (SCCG '09)*, 2009, pp. 95–102, <https://doi.org/10.1145/1980462.1980483>.
 - [35] T. Aila, S. Laine, Understanding the Efficiency of Ray Traversal on GPUs, *Proceedings of the Conference on High Performance Graphics 2009 (HPG '09)*, 2009, pp. 145–149, <https://doi.org/10.1145/1572769.1572792>.
 - [36] T. Aila, S. Laine, T. Karras, Understanding the Efficiency of Ray Traversal on GPUs – Kepler and Fermi Addendum, NVIDIA Corporation, 2012 Tech. Rep. NVR-2012-02.
 - [37] J. Mahovsky, B. Wyvill, Memory-Conserving Bounding Volume Hierarchies with Coherent Raytracing, *Comput. Graphics Forum* 25 (2) (2006) 173–182, <https://doi.org/10.1111/j.1467-8659.2006.00933.x>.
 - [38] T. Kim, B. Moon, D. Kim, S. Yoon, RACBVHs: Random-Accessible Compressed Bounding Volume Hierarchies, *IEEE Trans. Visual Comput. Graphics* 16 (2) (2010) 273–286, <https://doi.org/10.1109/TVCG.2009.71>.
 - [39] B. Segovia, M. Ernst, Memory Efficient Ray Tracing with Hierarchical Mesh Quantization, *Proceedings of Graphics Interface 2010 (GI '10)*, 2010, pp. 153–160.
 - [40] S. Keely, Reduced Precision for Hardware Ray Tracing in GPUs, *Proceedings of High Performance Graphics (HPG '14)*, 2014, pp. 29–40.
 - [41] K. Vaidyanathan, T. Akenine-Möller, M. Salvi, Watertight Ray Traversal with Reduced Precision, *Proceedings of High Performance Graphics (HPG '16)*, 2016, pp. 33–40.
 - [42] CUDA C Programming Guide, 2018, NVIDIA Corporation, Tech. Rep. PG-02928-001_v10.0.
 - [43] neighbor 0.1.0. <https://github.com/mphoward/neighbor>.
 - [44] CUB 1.8.0. <https://nvlabs.github.io/cub>.
 - [45] IEEE Standard for Floating-Point Arithmetic, IEEE Std. 754-2008, 2008. <https://doi.org/10.1109/IEEESTD.2008.4610935>.
 - [46] M.P. Howard, A.Z. Panagiotopoulos, A. Nikoubashman, Efficient mesoscale hydrodynamics: Multiparticle collision dynamics with massively parallel GPU acceleration, *Comput. Phys. Commun.* 230 (2018) 10–20, <https://doi.org/10.1016/j.cpc.2018.04.009>.
 - [47] C.L. Phillips, J.A. Anderson, S.C. Glotzer, Pseudo-random number generation for Brownian Dynamics and Dissipative Particle Dynamics simulations on GPU devices, *J. Comput. Phys.* 230 (19) (2011) 7191–7201, <https://doi.org/10.1016/j.jcp.2011.05.021>.
 - [48] A. Ahmed, R.J. Sadus, Solid-liquid equilibria and triple points of *n*-6 Lennard-Jones fluids, *J. Chem. Phys.* 131 (17) (2009) 174504, <https://doi.org/10.1063/1.3253686>.
 - [49] J.J. Potoff, A.Z. Panagiotopoulos, Critical point and phase behavior of the pure fluid and a Lennard-Jones mixture, *J. Chem. Phys.* 109 (24) (1998) 10914–10920, <https://doi.org/10.1063/1.477787>.
 - [50] A.Z. Panagiotopoulos, Molecular Simulation of Phase Coexistence: Finite-Size Effects and Determination of Critical Parameters for Two- and Three-Dimensional Lennard-Jones Fluids, *Int. J. Thermophys.* 15 (6) (1994) 1057–1072, <https://doi.org/10.1007/BF01458815>.
 - [51] J.D. Weeks, D. Chandler, H.C. Andersen, Role of Repulsive Forces in Determining Equilibrium Structure of Simple Liquids, *J. Chem. Phys.* 54 (12) (1971) 5237–5247, <https://doi.org/10.1063/1.1674820>.
 - [52] NVIDIA Turing GPU Architecture, 2018, NVIDIA Corporation, Tech. Rep. WP-09183-001_v01.